

BASIC JAVA NOTES (BEGINNER FRIENDLY)

1. INTRODUCTION TO JAVA

- Java is a high-level, object-oriented programming language.
- Platform independent (Write Once, Run Anywhere).
- Used for web apps, Android apps, enterprise applications.

2. JAVA FEATURES

- Object-Oriented
- Simple & Secure
- Portable
- Robust
- Multithreaded
- High Performance (JIT Compiler)

3. BASIC STRUCTURE OF A JAVA PROGRAM

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

4. DATA TYPES

Primitive:

- int (4 bytes)
- float (4 bytes)
- double (8 bytes)
- char (2 bytes)
- boolean (1 bit)
- byte, short, long

Non-primitive:

- String
- Arrays
- Classes

5. VARIABLES

- `int x = 10;`
- `String name = "John";`

6. OPERATORS

- Arithmetic: `+`, `-`, `*`, `/`
- Relational: `==`, `!=`, `<`, `>`
- Logical: `&&`, `||`, `!`

7. CONTROL STATEMENTS

IF-ELSE:

```
if (x > 10) {  
    System.out.println("Greater");  
} else {  
    System.out.println("Smaller");  
}
```

SWITCH:

```
switch(a) {  
    case 1: break;  
    default: break;  
}
```

8. LOOPS

FOR:

```
for (int i = 0; i < 5; i++) {}
```

WHILE:

```
int i = 0;
```

```
while(i < 5) { i++; }
```

DO-WHILE:

```
do { i++; } while(i < 5);
```

9. ARRAYS

```
int[] nums = {1, 2, 3};
```

```
String[] names = new String[5];
```

10. METHODS

```
public static int add(int a, int b) {
```

```
    return a + b;
```

```
}
```

11. OOP CONCEPTS

- Class & Object

- Encapsulation

- Inheritance

- Polymorphism

- Abstraction

Example Class:

```
class Student {
```

```
    String name;
```

```
    int age;
```

```
}
```

12. CONSTRUCTORS

```
class Person {
```

```
    Person() {
```

```
System.out.println("Constructor called");  
}  
}
```

13. INHERITANCE

```
class Animal {}  
class Dog extends Animal {}
```

14. POLYMORPHISM

Method Overloading:

```
void sum(int a, int b)  
void sum(double a, double b)
```

Method Overriding:

```
class A { void show(){} }  
class B extends A { void show(){} }
```

15. EXCEPTION HANDLING

```
try {  
    int x = 10/0;  
} catch(Exception e) {  
    System.out.println(e);  
}
```

16. FILE HANDLING (BASIC)

```
FileWriter fw = new FileWriter("data.txt");  
fw.write("Hello");  
fw.close();
```

17. COLLECTION FRAMEWORK

- List (ArrayList)
- Set (HashSet)

- Map (HashMap)

Example:

```
ArrayList list = new ArrayList<>();
```

18. FINAL KEYWORDS

- final variable → constant
- final method → cannot override
- final class → cannot extend

19. STATIC KEYWORD

```
static int count;  
static void print(){}
```

20. INTERFACE

```
interface Animal {  
    void sound();  
}
```

```
class Dog implements Animal {  
    public void sound(){  
        System.out.println("Bark");  
    }  
}
```

END OF BASIC NOTES